



Code intermédiaire

Sommaire

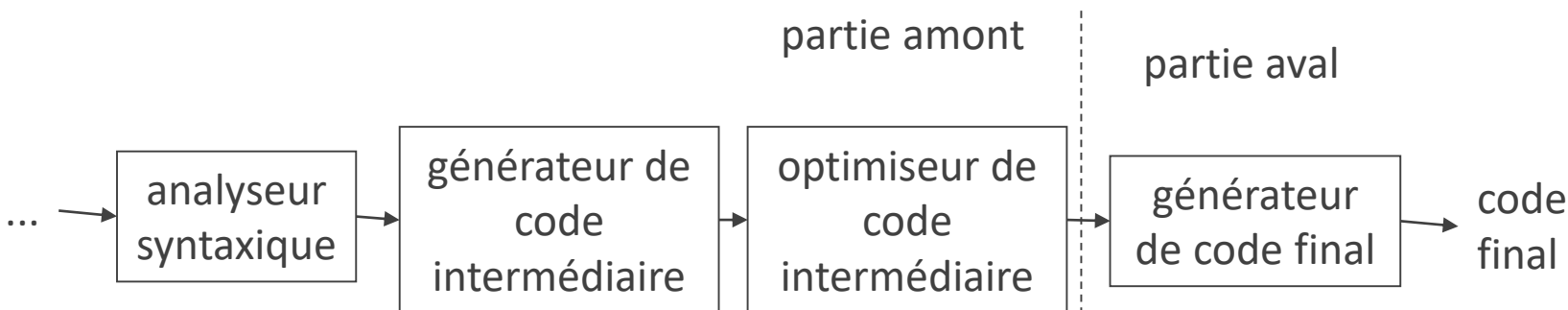
Arbre abstrait

[Code à trois adresses](#)

[Byte code](#)

[Éléments de tableaux](#)

Génération de code intermédiaire



Code intermédiaire

Exemples : arbre abstrait ; bytecode Java

Indépendant de l'architecture de la machine cible

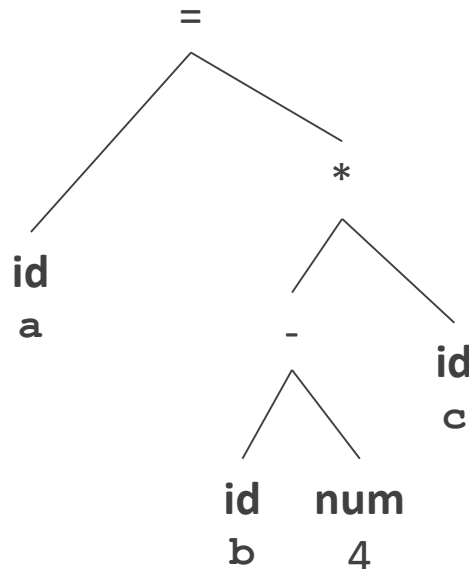
Pas de différence entre registres et accès mémoire

Séparer code intermédiaire et final permet de modulariser

- un générateur de code intermédiaire
- un générateur de code final (ou une machine virtuelle) dépendant de l'architecture de la machine cible

Code intermédiaire sous forme d'arbre abstrait

$a = (b - 4) * c$



Exemple : le langage GENERIC utilisé par gcc
 Chaque nœud correspond à une instruction
 L'ordre dans lequel il faut exécuter les instructions
 est donné par le parcours de l'arbre
 Parcours en profondeur (*depth-first*)
 Chaque nœud représentant un opérateur a pour
 fils les nœuds représentant les opérandes

Sommaire

[Arbre abstrait](#)

Code à trois adresses

Byte code

Éléments de tableaux

Code à trois adresses

```
tmp1 := $a  
tmp2 := $4  
tmp3 := b - tmp2  
tmp4 := tmp3 * c  
base[tmp1] := tmp4
```

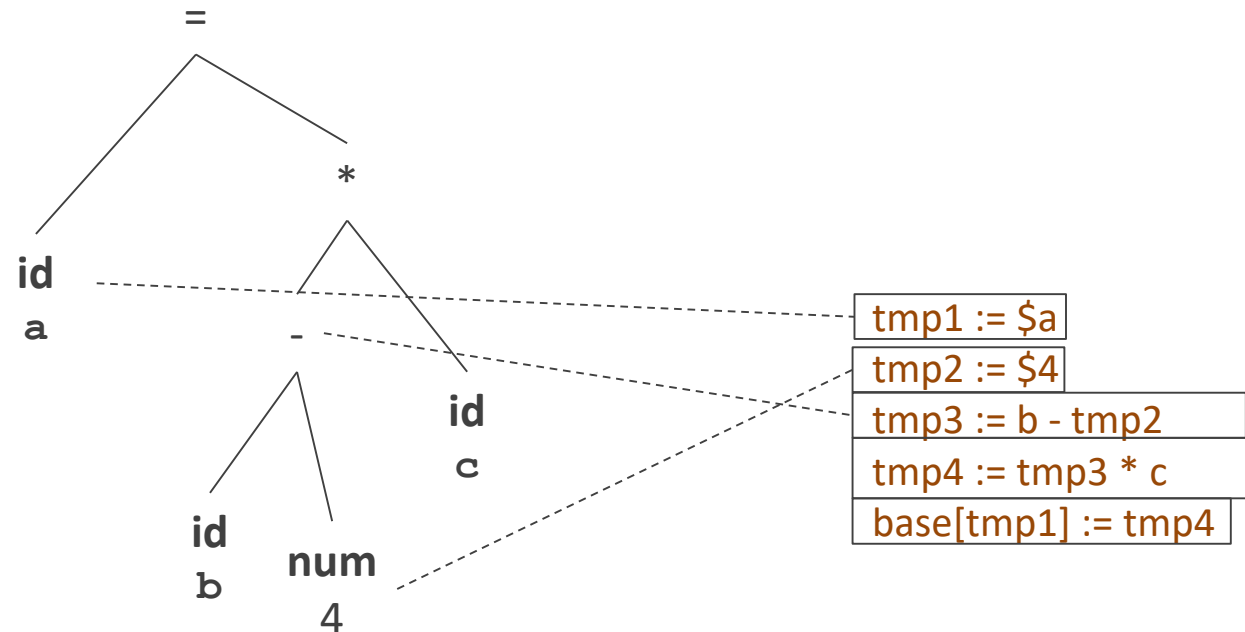
Séquence d'instructions

La séquence correspond à l'ordre de parcours des
nœuds d'un arbre abstrait

Beaucoup de variables temporaires

Presque toutes les valeurs utilisées doivent
pouvoir être calculées avant, et placées en
mémoire

Code à trois adresses et arbre abstrait



On suppose que l'adresse de `a` est une adresse relative
Chaque instruction à trois adresses peut être calculée à partir
d'un nœud de l'arbre abstrait et de ses fils

Code à trois adresses

```
tmp1 := $a
tmp2 := $4
tmp3 := b - tmp2
tmp4 := tmp3 * c
base[tmp1] := tmp4
```

Instructions de copie

```
x := y
x := $y
```

Pour les fonctions

```
call x, y
param x
return x
```

Forme générale pour les opérateurs : **$x := y \text{ op } z$**

où **op** est un opérateur. Les trois adresses sont celles de **x**, **y** et **z**.

Instructions d'affectation

```
x := y op z      x := op y
```

Par défaut, les opérandes sont des adresses et il y a déréférencement

Pour un opérande sans déréférencement :

```
x := $y op z      x := y op $z      x := $y op $z
```

Instructions de saut

```
inconditionnel goto L
conditionnel   if x relop y goto L
```

Instructions indicées

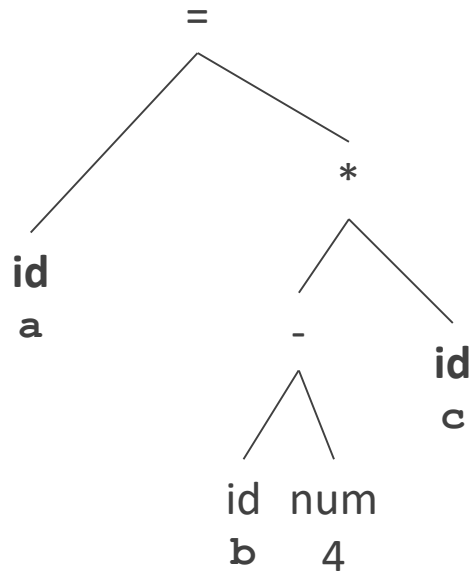
```
x := y [ z ]      x [ y ] := z
```


Représentation interne

Tableau à 4 colonnes

```
tmp1 := $a
tmp2 := $4
tmp3 := b - tmp2
tmp4 := tmp3 * c
base[tmp1] := tmp4
```

op	adr 1	adr 2	adr 3
:=	tmp1	\$a	
:=	tmp2	\$4	
-	tmp3	b	tmp2
*	tmp4	tmp3	c
[]:=	base	tmp1	tmp4



Traduire un arbre abstrait en code à trois adresses

$a = (b - 4) * c$

Parcours de l'arbre :

$tmp1 := \$a$

$tmp2 := \$4$

$tmp3 := b - tmp2$

$tmp4 := tmp3 * c$

$base[tmp1] := tmp4$

1.

:=	\$a	
----	-----	--

2.

b		
---	--	--

3.

:=	\$4	
----	-----	--

4.

-	2	3
---	---	---

5.

c		
---	--	--

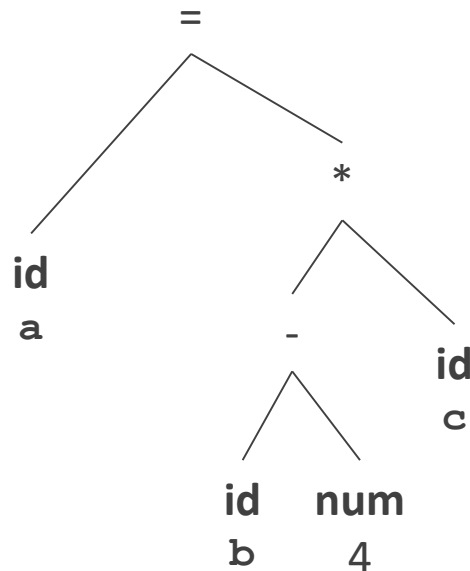
6.

*	4	5
---	---	---

7.

[]:=	1	6
------	---	---

Traduire un arbre abstrait en code à trois adresses



```

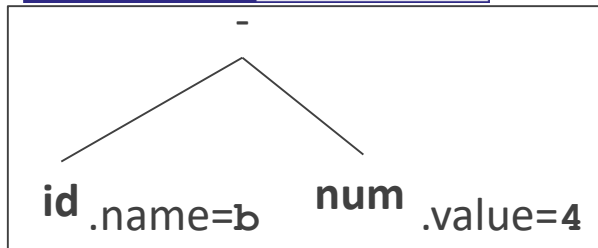
tmp1 := $a
tmp2 := $4
tmp3 := b - tmp2
tmp4 := tmp3 * c
base[tmp1] := tmp4
  
```

Principe

Le compilateur parcourt l'arbre abstrait en
profondeur

À chaque nœud de l'arbre abstrait il écrit du code
à trois adresses dans un fichier

L'ordre des instructions dans le fichier cible
correspond à l'ordre dans lequel on les écrit



nœud

pseudocode du compilateur

Spécifier des actions dans un parcours d'arbre

-

```

visit(-.firstchild); visit(-.secondchild);
-.place = newtemp();
write(-.place ' := ' -.firstchild.place '-' -.secondchild.place);
  
```

id

```

p = lookup(id.name);
if (p) id.place = p.place else semantic_error();
  
```

num

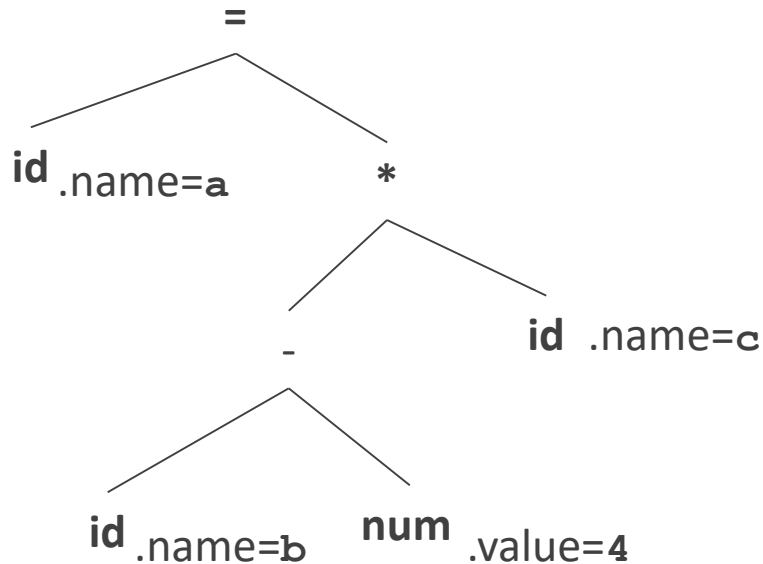
```

num.place = newtemp();
write(num.place ' := $ ' num.value);
  
```

Attributs des nœuds de l'arbre
.name : identificateur
.value : valeur d'une constante
.place : adresse

nœud	instructions de code intermédiaire
id	
num	tmp2 := \$4
-	tmp3 := b - tmp2

Spécifier des actions dans un parcours d'arbre



Attributs des nœuds de l'arbre
.name : identificateur
.value : valeur d'une constante
.place : adresse

nœud instructions de code interméd.

id

tmp1 := \$a

id

num

tmp2 := \$4

-

tmp3 := b - tmp2

*

tmp4 := tmp3 * c

=

base[tmp1] := tmp4

Traduire un arbre abstrait en code à trois adresses

Noeud Pseudocode du compilateur

id

```
id.place = newtemp(); p = lookup(id.name) ;
if (p) write(id.place ' :=$ ' p.place) else semantic_error();
```

=

```
visit_left(=.firstchild); visit(=.secondchild);
write('base[ ' =.firstchild.place ' ] :=' =.secondchild.place) ;
```

-

```
visit(-.firstchild); visit(-.secondchild);
-.place = newtemp() ;
write(-.place ' :=' -.firstchild.place ' - ' -.secondchild.place) ;
```

```
visit(*.firstchild); visit(*.secondchild);
*.place = newtemp() ;
write(*.place ' :=' *.firstchild.place ' * ' *.secondchild.place) ;
```

id

```
p = lookup(id.name) ;
if (p) id.place = p.place else semantic_error() ;
```

num

```
num.place = newtemp();
write(num.place ' :=$ ' num.value) ;
```

Exemples

```
tmp1 := $a
tmp2 := $4
tmp3 := b - tmp2
tmp4 := tmp3 * c
base[tmp1] := tmp4
```

Exercices sur les indirections

```
x := $k
z := y [ x ]
est équivalent à
x := $k
v := x + y
w := $0
z := w [ v ]
```

Instructions indicées

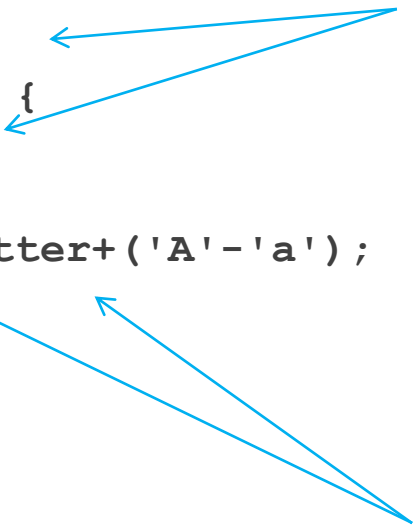
$x := y [z]$ $x [y] := z$

Pour un opérande sans déréférencement :

$x := \$y [z]$	$x := y [\$z]$	$x := \$y [\$z]$
$\$x [y] := z$	$x [\$y] := z$	$\$x [\$y] := z$
$x [y] := \$z$	$\$x [y] := \z	
$x [\$y] := \z	$\$x [\$y] := \$z$	

Adresses des variables

```
int uppercase;
int main(void) {
    int letter;
    letter='c';
    uppercase=letter+('A'-'a');
    return 0;
}
```



Pendant le traitement des déclarations

Décider les adresses des variables déclarées
Les sauvegarder dans la table des symboles

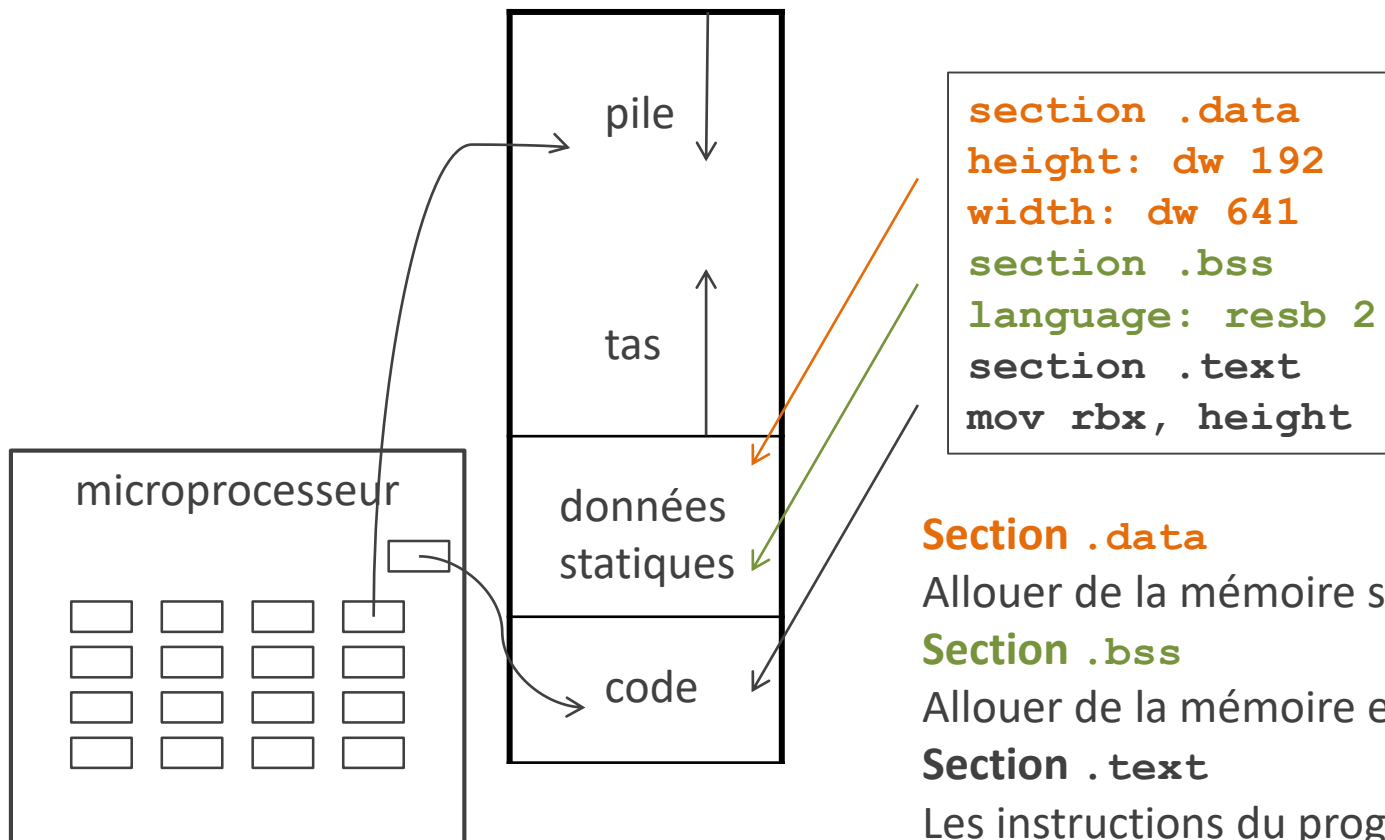
Avant de traduire les instructions

Préparer l'allocation mémoire pour les variables
Variables globales : avant de traduire les fonctions
Variables locales : avant de traduire le corps de la fonction

Pendant la traduction des instructions

Trouver les adresses des variables dans la table des symboles

Allouer de la mémoire statique



Section .data

Allouer de la mémoire statique et initialiser

Section .bss

Allouer de la mémoire et initialiser à 0

Section .text

Les instructions du programme

Allouer de la mémoire statique

```

section .data
height: dw 192
width: dw 641
my_vars: times 5 dd 0
section .bss
language: resb 2
my_statics: resd 5
section .text
mov rbx, height
    
```

Section .data

db, dw, dd, dq indiquent un nombre d'octets

Valeurs initiales

Pour multiplier ce nombre : **times**

Section .bss

resb, resw, resd, resq indiquent un nombre d'octets

L'entier multiplie ce nombre d'octets

Sommaire

Arbre abstrait

Code à trois adresses

Byte code

Éléments de tableaux

Byte code

iload b
ldc &4
isub
iload c
imul
istore a

Séquence d'instructions

Pile

Pour évaluer une expression, sauvegarde les
opérandes dans la pile

Byte code

```
; nasm
push qword [b]
push 4
```

```
pop rbx
pop rax
sub rax, rbx
push rax
```

```
push qword [c]
```

```
pop rbx
pop rax
imul rax, rbx
push rax
```

```
pop qword [a]
```

iload b

ldc &4

isub

iload c

imul

istore a

Les instructions pour les opérations empilent et dépilent automatiquement

a = (b - 4) * c

accès mémoire, **empiler**

accès zone de constantes, **empiler**

dépiler 2 fois, soustraction, **empiler**

empiler

dépiler 2 fois, multiplication, **empiler**

dépiler, accès mémoire

Code à trois adresses et byte code

$a = (b - 4) * c$

tmp1 := \$a

tmp2 := \$4

tmp3 := b - tmp2

tmp4 := tmp3 * c

base[tmp1] := tmp4

iload b

ldc &4

isub

iload c

imul

istore a

La pile du byte code remplace les
variables temporaires du code à
trois adresses

Sommaire

Arbre abstrait

Code à trois adresses

Byte code

Éléments de tableaux

Instructions de code à trois adresses pour les tableaux

```
; nasm  
mov rcx, qword [y]  
mov rdx, qword [z]  
mov rax, qword [rcx+rdx]  
mov qword [x], rax
```

```
; nasm  
mov rax, qword [x]  
mov rcx, qword [y]  
mov rdx, qword [z]  
mov qword [rax+rcx], rdx
```

x := y [z]

y contient l'adresse générale du tableau

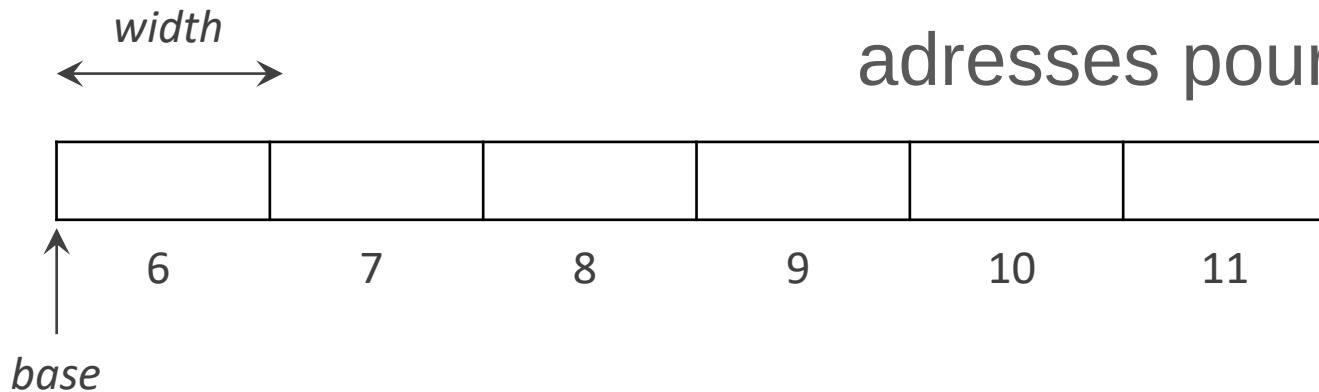
z contient l'adresse relative (*offset*) en octets d'un élément dans le tableau

x [y] := z

x contient l'adresse générale du tableau

y contient l'adresse relative d'un élément dans le tableau

Instructions de code à trois adresses pour les tableaux



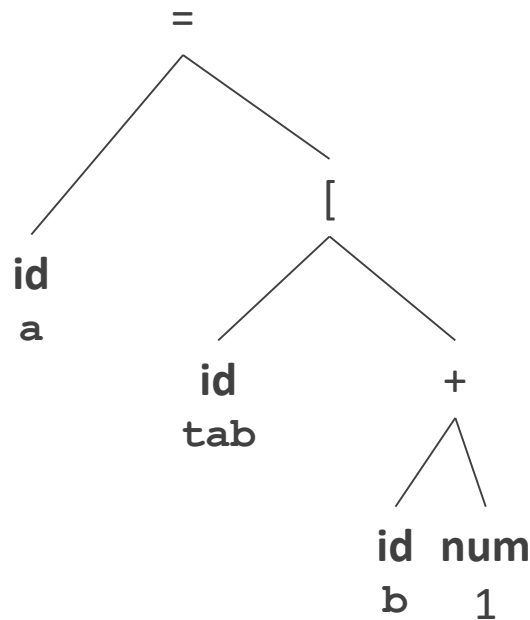
$x := y [z]$

$x [y] := z$

En langage C, $low = 0$

Différences avec un langage de haut niveau

- Pas d'expression à l'intérieur des crochets, alors qu'en C on peut avoir $a[i+2]$
- Si on incrémente z de 1, $y[z]$ est 1 octet plus loin
Si on incrémente i de 1, $a[i]$ est 1 case plus loin
Taille d'une case : $width$ octets
- Le premier élément n'est pas forcément $z[0]$
Borne inférieure des indices : low



En langage C, l'expression se réduit à

$p.base + i \cdot p.width$

Traduire un arbre abstrait en code à trois adresses

Champs dans la table des symboles

$p.width$: taille mémoire de chaque case du tableau

$p.base$: adresse de la première case

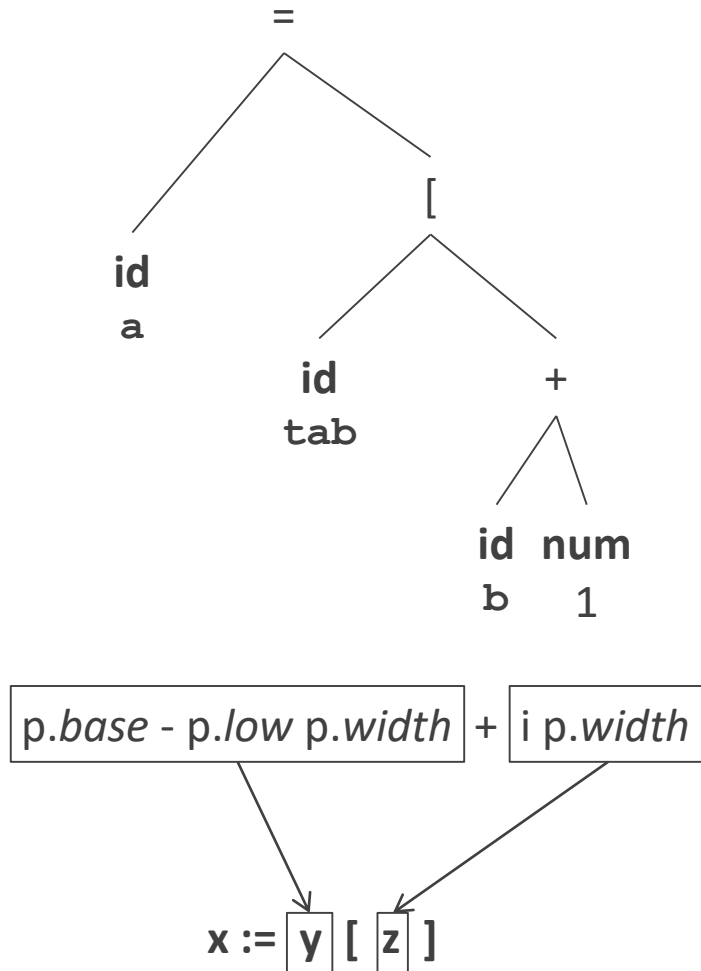
$p.low$: indice de la première case

Adresse de **tab[i]** $p.base + (i - p.low) \cdot p.width$

Un programme peut être compilé une fois et exécuté des milliers de fois

Faire la plus grande partie possible du calcul à la compilation

Adresse d'un élément de tableau



Champs dans la table des symboles

p.width : taille mémoire de chaque case du tableau

p.base : adresse de la première case

p.low : indice de la première case

p.const : l'adresse du tableau calculable à la compilation, $p.base - p.low \cdot p.width$

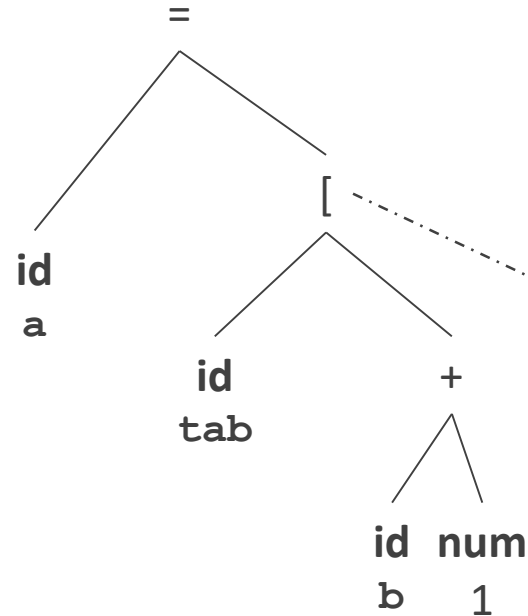
Adresse de `tab[i]` $p.base + (i - p.low) \cdot p.width$

Un programme peut être compilé une fois et exécuté des milliers de fois

Faire la plus grande partie possible du calcul à la compilation

Quelle partie peut être calculée à la compilation ?

Traduire un arbre abstrait en code à trois adresses



```

tmp1 := $a
tmp2 := $1
tmp3 := b + tmp2
tmp4 := $ p.width * tmp3
tmp5 := $ p.const [tmp4]
base[tmp1] := tmp5
  
```

Nœud [

```

p = lookup([.firstchild.name) ; if (p==0) semantic_error();
visit([.secondchild); [.offset = newtemp();
write([.offset ' := $' p.width '*' [.secondchild.place);
[.place = newtemp(); write( [.place ' := $' p.const '[' [.offset ' ] ' ) ;
  
```

Tableaux à plusieurs dimensions

$a[i][j]$	$j = 0$	$j = 1$	$j = 2$
$i = 0$	$base$	$base + w$	$base + 2w$
$i = 1$	$base + 3w$	$base + 4w$	$base + 5w$

adresses

basses



hautes

$a[0][0]$

$a[0][1]$

$a[0][2]$

$a[1][0]$

$a[1][1]$

$a[1][2]$

Dans la plupart des langages, dont C, les éléments sont rangés ligne par ligne

Quand on passe de chaque case du tableau à sa voisine dans la mémoire, c'est le dernier indice qui varie le plus vite

En Fortran : colonne par colonne

Tableaux à 2 dimensions

$a[i_1][i_2]$	$i_2 = 1$	...	$i_2 = n_2$
$i_1 = 1$	$base$	...	$base + (n_2 - 1)w$
...	...		...
$i_1 = n_1$	$base + (n_1 - 1)n_2w$	...	$base + (n_1n_2 - 1)w$

Adresse de $a[i_1][i_2]$

Chaque ligne contient n_2 éléments, où n_2 est le nombre des valeurs possibles de i_2

Les lignes avant la i_1 contiennent au total $(i_1 - low_1) n_2$ éléments

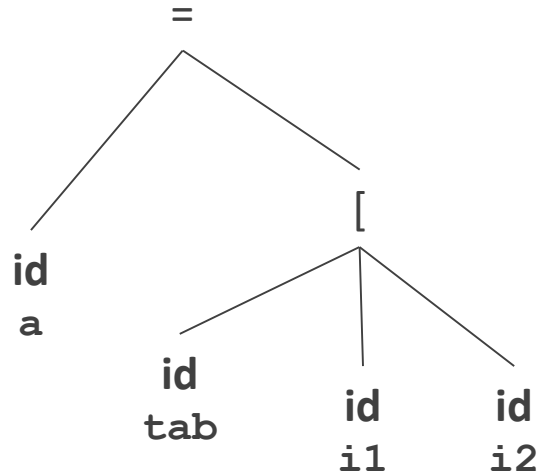
Donc $base + ((i_1 - low_1) n_2 + i_2 - low_2) w$

Quelle partie peut être calculée à la compilation ?

$$base - (low_1 n_2 + low_2) w + (i_1 n_2 + i_2) w$$

$x := y [z]$

Tableaux à 2 dimensions



$$base - (low_1 n_2 + low_2) w + (i_1 n_2 + i_2) w$$

$x := y [z]$

Champs dans la table des symboles

$p.w$: taille mémoire de chaque case du tableau

$p.base$: adresse de la première case

$p.low(1)$: premier indice à la première dimension

$p.low(2)$: premier indice à la 2^e dimension

$p.extent(2)$: n_2 , nombre de valeurs possibles à la 2^e dimension

$p.const = base - (low_1 n_2 + low_2) w$

On n'a pas besoin du nombre de valeurs possibles du 1^{er} indice

```
int process(int area[][WIDTH]) ;
```

Tableaux à k dimensions

Adresse de $a[i_1][i_2]$: $base + ((i_1 - low_1) n_2 + i_2 - low_2) w$

On généralise à k dimensions

$base + ((...(((i_1 - low_1) n_2 + i_2 - low_2) n_3 + i_3 - low_3)...) n_k + i_k - low_k) w$

Une partie peut être calculée à la compilation

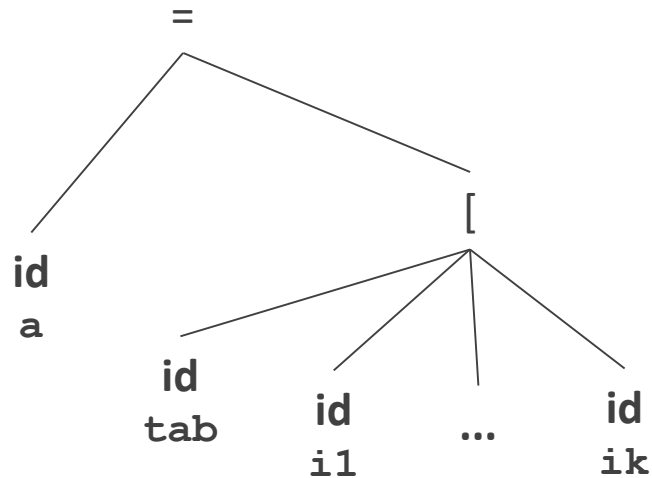
$base - ((...((low_1 n_2 + low_2) n_3 + low_3)...) n_k + low_k) w + ((...((i_1 n_2 + i_2) n_3 + i_3)...) n_k + i_k) w$

Le reste doit être calculé à l'exécution (adresse relative)

Le compilateur produit du code qui le calcule :

$$\begin{aligned} e_1 &= i_1 \\ e_d &= e_{d-1} n_d + i_d \quad \text{pour } 1 < d \leq k \\ offset &= e_k w \end{aligned}$$

Tableaux à k dimensions



Champs dans la table des symboles

$p.w$: taille mémoire de chaque case du tableau

$p.base$: adresse de la première case

$p.low(1)$: premier indice à la première dimension

...

$p.low(k)$: premier indice à la k^e dimension

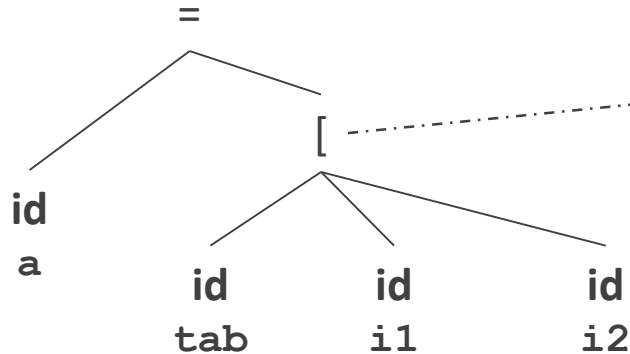
$p.extent(2)$: n_2 , nombre de valeurs, 2^e dimension

...

$p.extent(k)$: n_k , nombre de valeurs, k^e dimension

$p.const$

```
int process(int space[][WIDTH][DEPTH]) ;
```



tmp1 := \$a

tmp2 := i1

tmp3 := tmp2 * \$ p.extent(2)

tmp2 := tmp3 + i2

tmp4 := \$ p.width * tmp2

tmp5 := \$ p.const [tmp4]

base[tmp1] := tmp5

Noeud [

```

p = lookup([.firstchild.name) ; if (p==0) semantic_error();
[.index = newtemp() ; [.subexp = newtemp() ;
visit([.secondchild); write([.index ' := ' [.secondchild.place) ;
child = [.secondchild;
for (dim=2 to k)

```

```

write([.subexp ' := ' [.index ' * ' $' p.extent(dim));

```

```

child = child.next_sibling; visit(child); write([.index ' := ' [.subexp ' + ' child.place) ;

```

```

[.offset = newtemp(); write([.offset ' := ' $' p.width ' * ' [.index);

```

```

[.place = newtemp(); write( [.place ' := ' $' p.const ' [ ' [.offset ' ] ' ) ;

```

Merci

CONTACT

ERIC LAPORTE

+33 1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR